

UADE

PROGRAMACIÓN I

Lic. Julia Monasterio



UADE

Maria Julia Monasterio

- Lic en Sistemas de Información
- Docente en UADE, UB y UAI
- Project Manager – Grid Dynamics
- Docente en Educación IT
- Master en IA y Machine Learning

Mail: marmonasterio@uade.edu.ar



UADE

Sobre ustedes

- Preguntas disparadoras:
 - ¿Trabajan en el mundo IT?
 - ¿De donde son?
 - Año de la carrera

UADE

Aspectos generales

- LUGAR: Aula 707 – Monserrat
- Día/Horario: Jueves 13:30 a 17:30
- 75% de Asistencia
- NO PROMOCIONABLE



- **Unidad I: Repaso de matrices: estructura, operaciones y uso en funciones**
- **Unidad II: Implementación y gestión de proyectos con Git**
- **Unidad III: Listas avanzadas, cadena de caracteres y expresiones regulares**
- **Unidad IV: Manipulación avanzadas de diccionarios, tuplas y conjuntos**
- **Unidad V: Excepciones y pruebas unitarias**
- **Unidad VI: Archivos**
- **Unidad VII: Recursividad**



UADE

FECHAS IMPORTANTES

- **26/3 PRIMER PRE-ENTREGA: DOCUMENTO DE ALCANCE Y REPOSITORIO INICIADO**
- **02/04 FERIADO**
- **23/04 ENTREGA 40%**
- **30/4 PRIMER PARCIAL**
- **18/06 SEGUNDO PARCIAL**
- **25/06 ENTREGA DEL 100%**
- **02/07 RECUPERATORIO Y FINAL ADELANTADO**
- **09/07 FERIADO**
- **16/07 FINAL DESDOBLADO**
- **23/07 FINAL REGULAR**



UADE

Modalidad de evaluación

- Evaluación Individual
- Proyecto Cuatrimestral por etapas: el proyecto cuatrimestral tiene como objetivo principal que los estudiantes apliquen los conocimientos adquiridos en Programación I mediante el desarrollo de un proyecto cercano a una implementación laboral real.



UADE

Desarrollo del proyecto

- Los estudiantes deberán proponer un tema (puede ser un juego, programa de cálculo, programa de gestión, o cualquier otro tema) sujeto a la aprobación del docente.
- Los proyectos se programaran con el lenguaje Python y deben cubrir todos los temas del programa de la asignatura
- Los estudiantes se organizarán en equipos de 3 o 4 miembros. NO SE PUEDE REALIZAR EL TP DE FORMA INDIVIDUAL

UADE

Entregas del proyecto

- 1er Entrega: Se espera que en esta entrega se avance un 40% del trabajo final
- 2da Entrega: Se espera que en esta entrega se avance un 100% del trabajo.

UADE

Evaluación del proyecto

- Se evaluarán los siguientes aspectos:
 - Funcionamiento del programa
 - Cumplimiento de los objetivos planteados
 - Técnicas de programación y estrategias de resolución del problema
 - Prolijidad y claridad del código y de la interfaz de usuario
 - Se valorará el apoyo visual que harán en la presentación

UADE

Evaluación del proyecto

La calificación final recibida por cada integrante dependerá no solo del trabajo presentado sino también de la exposición ejercida en forma individual, así como las preguntas que se le realizarán a cada estudiante posteriormente a la exposición del trabajo.

Por esta razón, pueden tener notas diferentes miembros del mismo grupo.

UADE

Evaluación de avances **TODAS LAS CLASES**

UADE

Defensa de los entregables

Metodología para la defensa:

- Se deberá ejecutar el código sin errores, recorriendo todas las opciones o casuísticas que requieran
- Deberán contar cuales fueron las técnicas utilizadas y las estrategias de resolución, como así tambien los desafíos que encontraron durante el proceso. NO ES OBJETIVO LEER TODAS LAS LINEAS DE CODIGO.

UADE

Defensa de los entregables

Metodología para la defensa:

- Todos los miembros del equipo deberán exponer parte del trabajo.
- Deben tener un soporte visual

UADE

Últimas aclaraciones

- NO SE ACEPTARAN TRABAJOS QUE INCLUYAN FUNCIONES O HERRAMIENTAS DE PYTHON TRATADOS EN CLASE (SALVO QUE EXISTA UN PREVIO PEDIDO Y AUTORIZACIÓN POR PARTE DEL DOCENTE)
- EL TRABAJO PARA EXPONER DEBERÁ SER EL MISMO QUE SE DESIGNO EL DÍA DE LA FECHA DE INICIO (SEGUNDA CLASE)
- SI UN PARTICIPANTE NO TIENE COMMITS EN GIT, SE CONSIDERARÁ DESAPROBADO

Clase 1 - Temario

- Introducción de la materia
- Repaso de funciones y listas
- Matrices



UADE

INTRODUCCIÓN

- En la materia continuaremos trabajando en el lenguaje de programación Python
- Como editor podrán utilizar el que prefieran Thony o Visual Studio Code.



```
print(results.keys())
# Use the json module to load the JSON data from the file
theJSON = json.load(f)

# now we can access the contents of the JSON file
if "title" in theJSON["metadata"]:
    print(theJSON["metadata"]["title"])

# output the number of events, also the magnitude greater than 4.0
count = theJSON["metadata"]["count"]
print(str(count) + " events recorded")

# for each event, print the place where it occurred
for i in theJSON["features"]:
    print(i["properties"]["place"])
    print("\n")

# print the events that only have a magnitude greater than 4.0
for i in theJSON["features"]:
    if i["properties"]["mag"] >= 4.0:
        print("%2.1f" % i["properties"]["mag"], i["properties"]["place"], "\n")

# print only the events where at least 1 person reported feeling something
print("Events that were felt:")
for i in theJSON["features"]:
    if i["properties"]["felt"] > 0:
        print("%2.1f" % i["properties"]["mag"], i["properties"]["place"], i["properties"]["felt"], "\n")
```


UADE

PYTHON

LENGUAJE DE ALTO NIVEL.
MULTIPARADIGMA

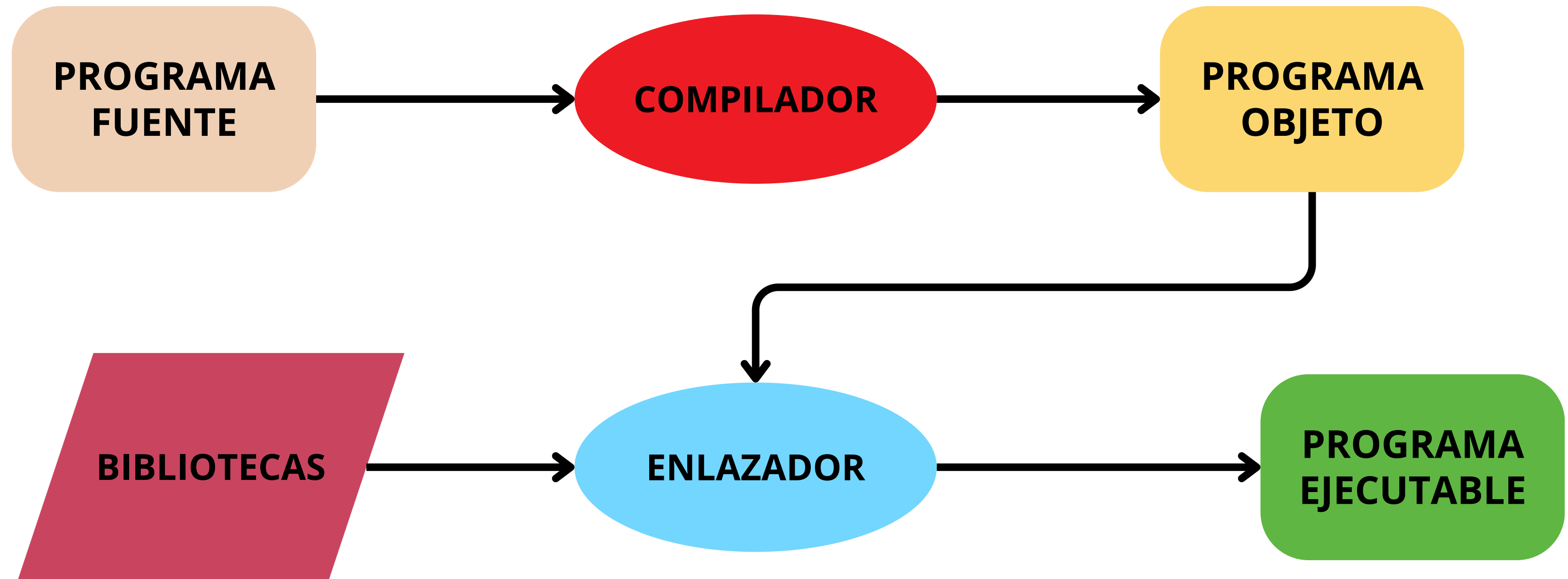
ES INTERPRETADO. NO SE
COMPILA

ES PORTABLE

ES CASESENSITIVE

UADE

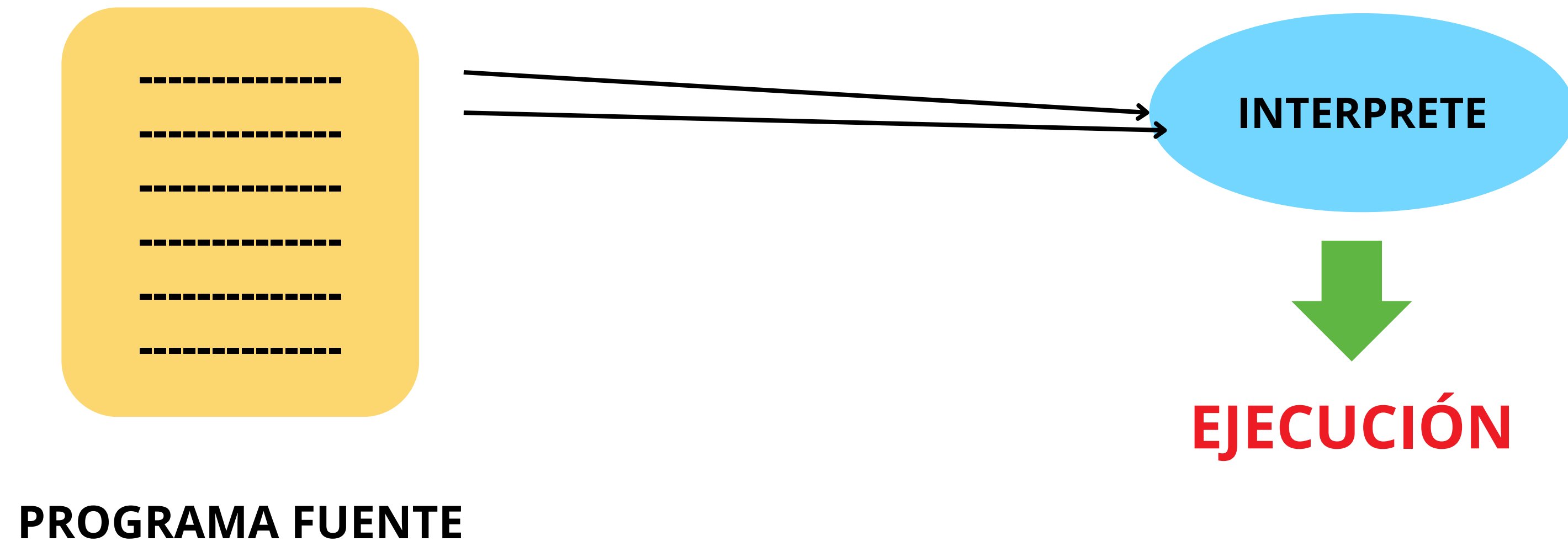
COMPILADOR



UADE

INTERPRETE

Traduce una por una las líneas del programa fuente, ejecutándolas inmediatamente



REPASO DE LISTAS

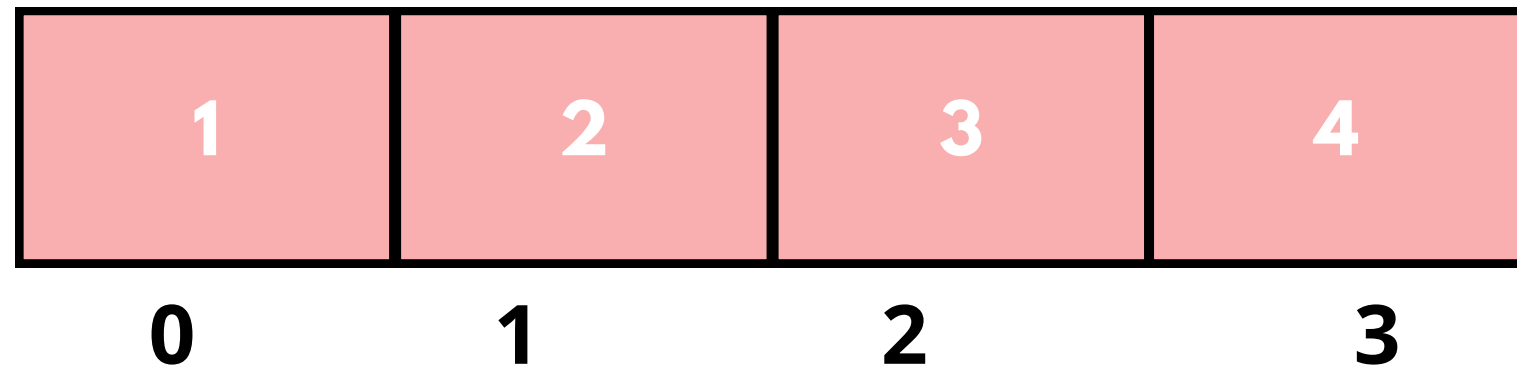
- Las listas son un tipo de contenedor que se usan para almacenar conjuntos de elementos relacionados del mismo o de distintos tipos
- Para crear una lista, simplemente hay que encerrar una secuencia de elementos separados por coma entre corchetes

REPASO DE LISTAS



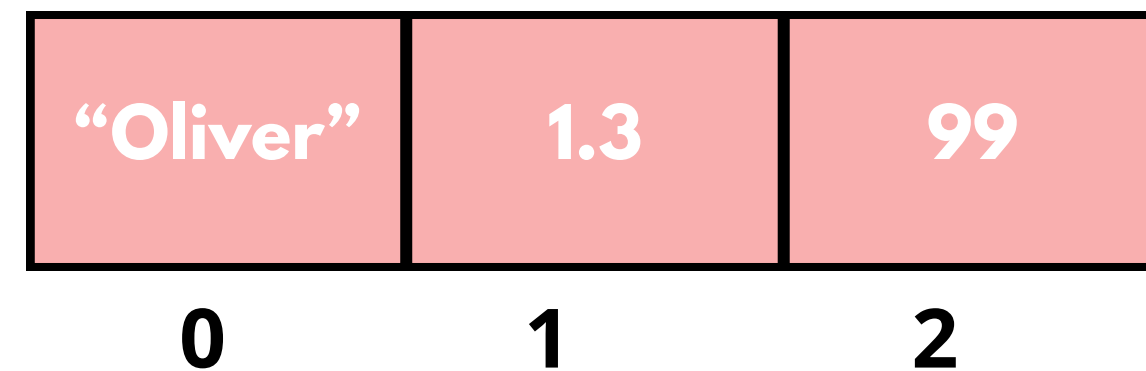
CREAR UNA LISTA CON
NÚMEROS

```
numeros=[1,2,3,4]
```



CREAR UNA LISTA CON
DATOS VARIADOS

```
listaVariada=['Oliver',1.3,99]
```

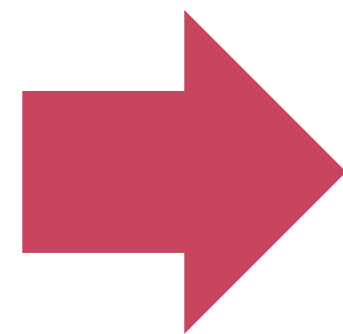


ACCEDER A UNA LISTA

Para acceder a un elemento de una lista se utilizan los índices. Un índice es un número entero que indica la posición de un elemento en una lista.

El primer elemento de una lista siempre comienza en el índice 0.

```
listaVariada=['Oliver',1.3,99]  
  
print(listaVariada[0])  
print(listaVariada[1])  
print(listaVariada[2])
```



```
Oliver  
1.3  
99
```

ERRORES



ERROR AL ACCEDER A UN ÍNDICE NO EXISTENTE

```
listaVariada=['Oliver',1.3,99]  
print(listaVariada[3])
```

```
print(listaVariada[3])
```

```
~~~~~^^^
```

```
IndexError: list index out of range
```



ERROR AL ENVIAR UN ÍNDICE NO NUMÉRICO

```
listaVariada=['Oliver',1.3,99]  
print(listaVariada[1.3])
```

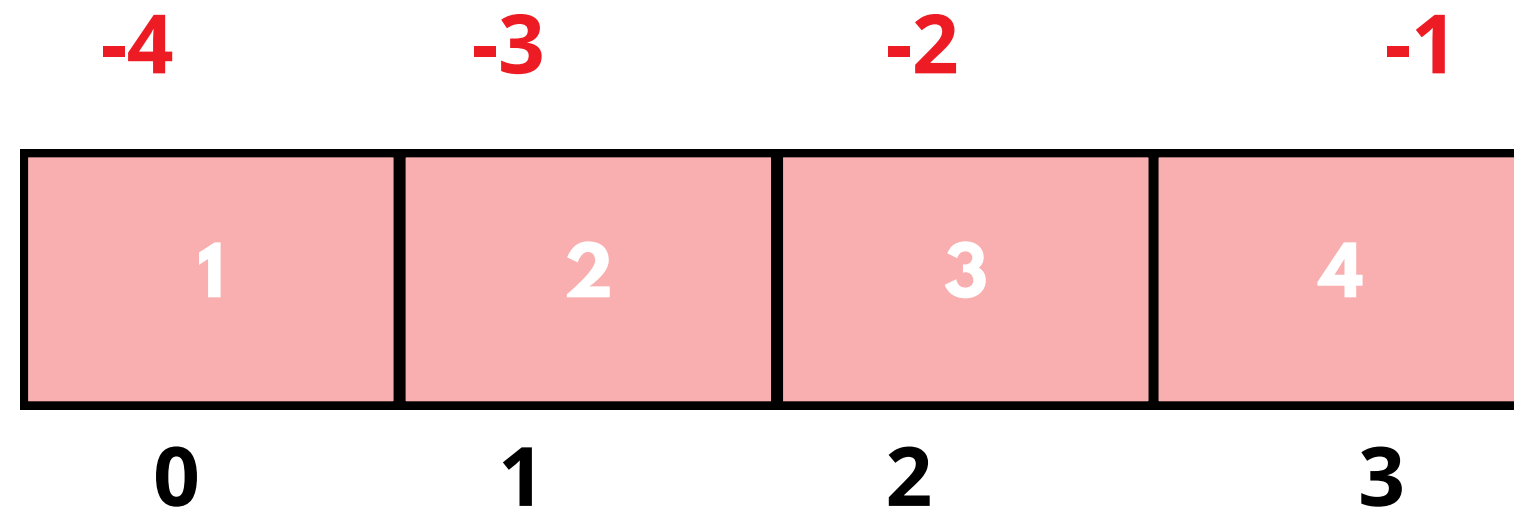
```
print(listaVariada[1.3])
```

```
~~~~~^^^
```

```
TypeError: list indices must be integers or slices, not float
```

ACCEDER A UNA LISTA CON ÍNDICE NEGATIVO

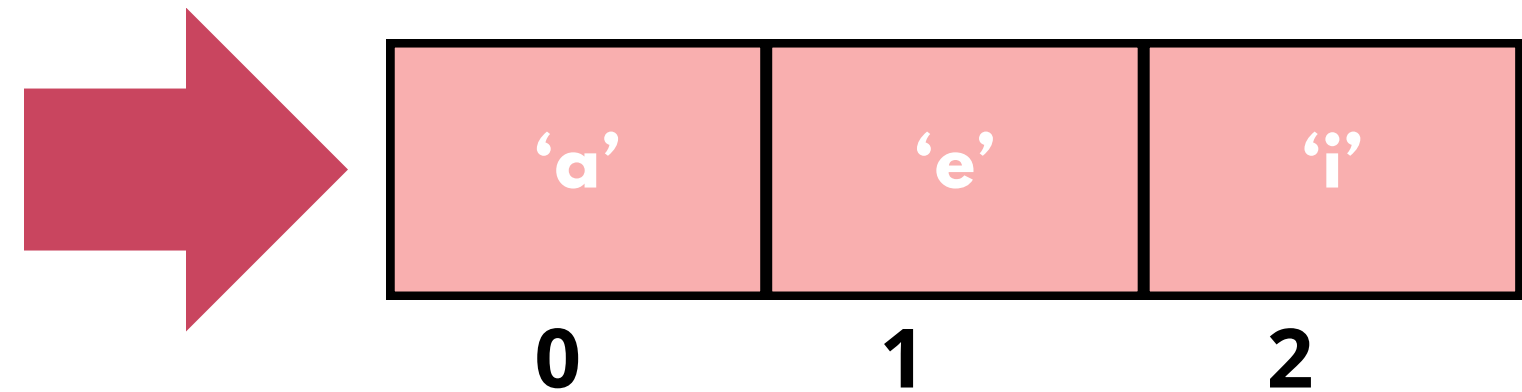
- En Python está permitido usar índices negativos para acceder a elementos de una secuencia.
- El índice -1 hace referencia al último elemento de la secuencia, -2 al penúltimo, y así sucesivamente



AÑADIR ELEMENTOS A UNA LISTA

- En una lista, sus elementos pueden ser modificados (se pueden añadir nuevos ítems, actualizar o eliminar)
- Para añadir un nuevo elemento al final se utiliza el método `append()`

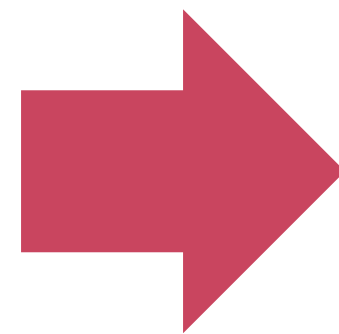
```
vocales=['a']  
  
vocales.append('e')  
vocales.append('i')
```



CONCATENAR LISTAS

También es posible utilizar el operador de concatenación + para unir dos listas en una sola. El resultado es una nueva lista con los elementos de ambas

```
lista1=[1,2,3]  
lista2 =[4,5,6]  
  
listaUnida = lista1 + lista2  
  
print(listaUnida)
```



```
[1, 2, 3, 4, 5, 6]
```

MODIFICAR ELEMENTOS DE UNA LISTA

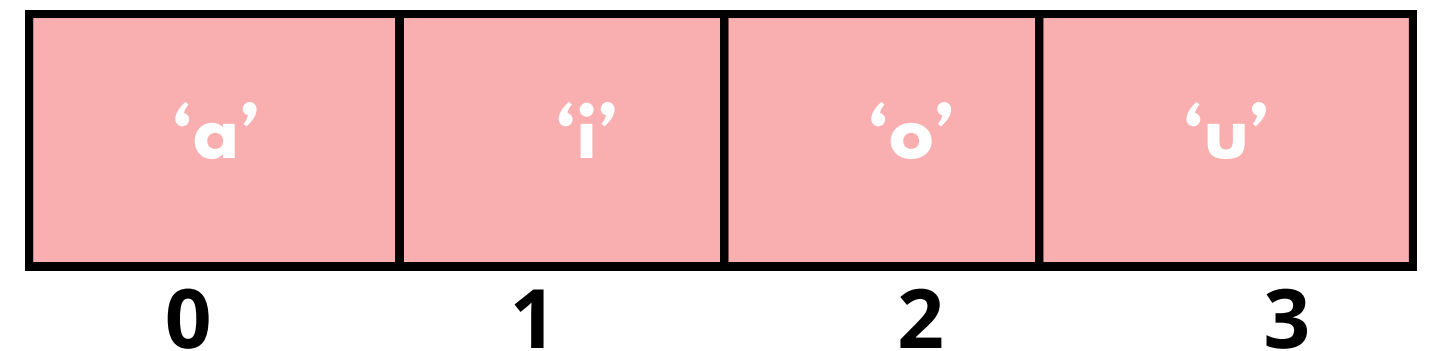
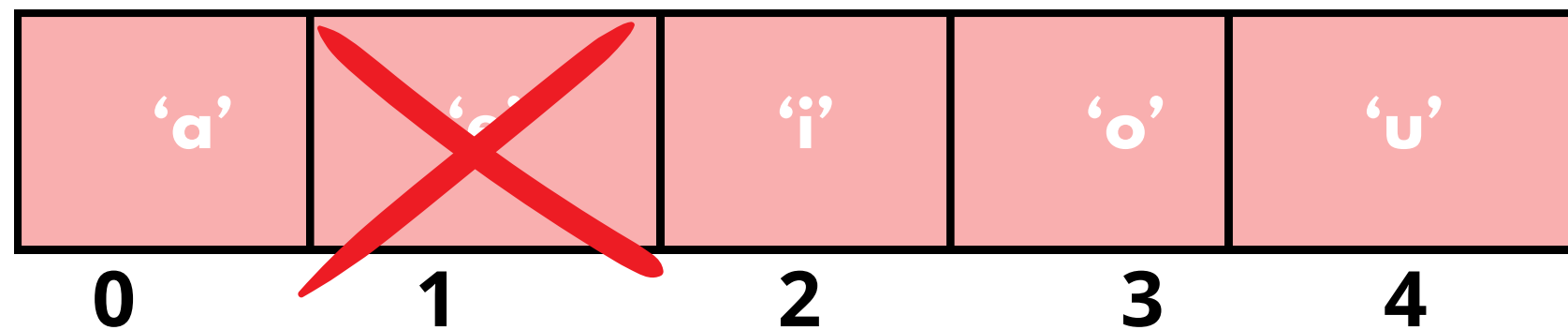
- Para modificar un elemento de una lista, se utiliza el operador de **asignación** “=”, que permite reemplazar el valor actual del elemento en la lista por otro valor nuevo
- Lo único que es necesario conocer es el **índice del elemento** que se quiere modificar, ya que es la referencia que se utiliza para ubicar el elemento específico en la lista

```
vocales=['o','o','o','o','u']  
vocales[0]='a'  
vocales[1]='e'  
vocales[2]='i'
```


ELIMINAR UN ELEMENTO DE UNA LISTA

Para eliminar un elemento de una lista, se puede realizar de varias formas. Con la sentencia `del` se puede eliminar un elemento a partir de su índice:

```
vocales=['a','e','i','o','u']  
del vocales[1] # Elimina el segundo elemento de la lista
```



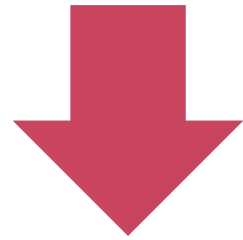
ELIMINAR UN ELEMENTO DE UNA LISTA

- También se puede usar los métodos `remove()` y `pop([i])`
- Con `remove()` eliminamos la primera ocurrencia que se encuentra del elemento en una lista.
- Con `pop([i])` obtiene el elemento cuyo índice sea igual a `i` y lo elimina de la lista. Si no se especifica ningún índice, recupera y elimina el último elemento

EJEMPLOS

✓ EJEMPLO REMOVE

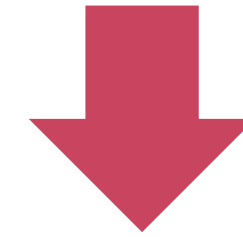
```
vocales=['a','e','i','o','u','a']  
  
vocales.remove('a') #Elimina la primer ocurrencia de a  
  
print(vocales)
```



```
['e', 'i', 'o', 'u', 'a']
```

✓ ERROR AL ENVIAR UN ÍNDICE NO NUMÉRICO

```
vocales=['a','e','i','o','u','a']  
  
vocales.pop()#Elimina el ultimo elemento  
vocales.pop(4) #Elimina el elemento en la posicion 4  
  
print(vocales)
```

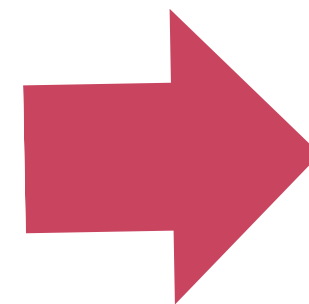


```
['a', 'e', 'i', 'o']
```

ELIMINAR TODOS LOS ELEMENTOS DE UNA LISTA

- El método **clear()** se utiliza para eliminar todos sus elementos de una lista.
- Al llamar a este método, todos los elementos se eliminan, dejándola vacía
- Esto puede ser útil cuando se necesita limpiar una lista existente para reutilizarla, en lugar de crear una nueva lista vacía

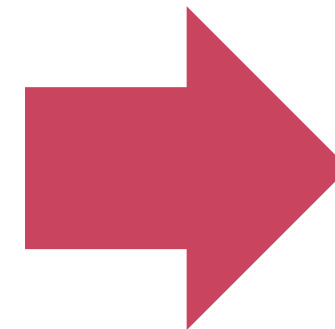
```
vocales=['a','e','i','o','u','a']  
  
vocales.clear()  
  
print(vocales)
```



LONGITUD DE UNA LISTA

- Para conocer la longitud de una lista se utiliza la función **len()**. Esta función devuelve el número de elementos de una lista.

```
vocales=['a','e','i','o','u','a']  
print(len(vocales))  
  
palabras=['hola','mundo','python']  
print(len(palabras))  
  
vacía=[]  
print(len(vacía))
```

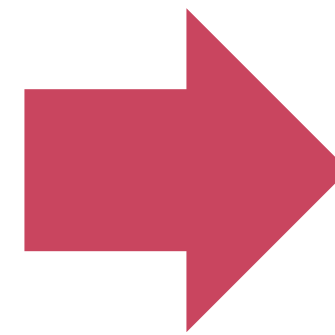


6
3
0

RECORRER UNA LISTA

- Es posible hacer un recorrido de las listas usando los índices y en conjunto con la función len()

```
vocales=['a','e','i','o','u','a']  
  
for i in range(len(vocales)):  
    print(vocales[i])
```



a
e
i
o
u
a

COMO SABER SI UN ELEMENTO ESTA EN UNA LISTA

- Para determinar la inclusión de un elemento dentro de una lista, se emplea el operador de pertenencia `in` o su opuesto `not in`
- Si el elemento en cuestión esta presente en la lista devuelve `True`, de lo contrario retorna `False`

IN

```
frutas=['manzana','pera','uva','sandia','melon']

if 'manzana' in frutas:
    print("El elemento esta presente")
else:
    print("El elemento no esta presente")
```

NOT IN

```
frutas=['manzana','pera','uva','sandia','melon']

if 'nuez' not in frutas:
    print("La nuez no esta en la lista de frutas")
else:
    print("La nuez esta presente en la lista de frutas")
```

EMPAQUETADO/ DESEMPAQUETADO



EMPAQUETADO

- Consiste en asignar un conjunto de variables o constantes a una lista

```
n1=10  
n2=11  
n3=14  
lista=[n1,n2,n3]
```



DESEMPAQUETADO

- Consiste en asignar los elementos de la lista a un conjunto de variables

```
dias=["Lunes","Martes","Miercoles"]  
variable1, variable2,variable3=dias
```


EMPAQUETADO/ DESEMPAQUETADO

POSIBLES ERRORES

- La cantidad de variables debe ser igual a la cantidad de elementos que tiene la lista, sino arroja error.

```
dias=["Lunes","Martes","Miercoles"]  
variable1, variable2,variable3,variable4=dias
```

```
1 - Lunes Noche (Clase 1\Practica tecnica\l1istas.py , line 3, in
    variable1, variable2,variable3,variable4=dias
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
ValueError: not enough values to unpack (expected 4, got 3)
```

```
dias=["Lunes","Martes","Miercoles"]
print(dias)
variable1,variable2 =dias
```

```
variable1,variable2 =dias
^^^^^^^^^^^^^^^^^^
ValueError: too many values to unpack (expected 2)
D:\C++\Hacer\1\lib\OpenDrive\Desktop\Compilador\HAF5\A~
```

REPLICAR LISTAS

- Permite crear una lista que contenga n veces la lista original

```
lista1=[1,2,3]  
  
lista2 = lista1*4  
  
print(lista2)
```

```
[1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3]
```

REPASO DE FUNCIONES

- Se divide el programa en muchos componentes o bloques pequeños autónomos llamados módulos, que son manejables, lógicos y funcionales.
- El módulo es un bloque de sentencias que resuelve un problema particular. Cada módulo contiene todo lo necesario para cumplir con su propia funcionalidad y se puede editar o modificar si que se vea afectado el resto del proyecto

CODIGO MODULARIZADO VS NO MODULARIZADO

NO MODULARIZADO

```
num1=10
num2=20
suma=num1+num2

print("La suma es ",suma)

num3=30
num4=40
suma2=num3+num4
print("La suma es",suma2)
```

MODULARIZADO

```
def sumar(a,b):
    suma=a+b
    print("La suma es: ",suma)

num1=10
num2=20
sumar(num1,num2)
num3=30
num4=40
sumar(num3,num4)
```

UADE

IDENTIFICACIÓN

UNICIDAD

REPRESENTATIVIDAD

NOMENCLATURA
CAMEL_CASE Y SNAKE

EVITAR PALABRAS
CLAVES Y CARACTERES
ESPECIALES

IMPORTANTE



FUNCIÓN ESPECÍFICA

Los módulos en programación deben adherirse a un principio fundamental: tener una función específica y no excederse de sus responsabilidades designadas



RESPONSABILIDAD ÚNICA

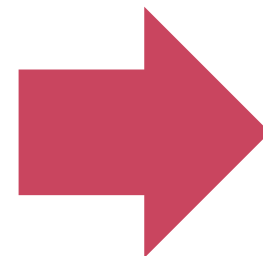
Este principio de responsabilidad única promueve la claridad, la eficiencia y la escalabilidad del código, lo que resulta fundamental en el desarrollo de software robusto y de calidad

DOCSTRING

- La cadena docstring no es obligatoria pero sirve para documentar la función
- Debe especificar que hace y no como lo hace
- Va encerrada entre tres juegos de comillas dobles o simples
- Este texto aparece al escribir `help(nombre_funcion)`

```
def areaTriangulo(base, altura):  
    """Objetivo: calcular el area de un triangulo  
    Parametros: base y altura del triangulo  
    Salida: resultado del area"""  
    return base * altura / 2
```

```
help(areaTriangulo)
```



```
Help on function areaTriangulo in module __main__:  
  
areaTriangulo(base, altura)  
    Objetivo: calcular el area de un triangulo  
    Parametros: base y altura del triangulo  
    Salida: resultado del area
```

MATRICES

- Son una estructura de datos de dos dimensiones de elementos contiguos, relacionados por un mismo nombre, donde los elementos son distinguidos por dos índices (normalmente denominado como fila y columna)
- Tanto las filas como las columnas, siempre empieza en el índice 0, y terminan en el índice (tamaño -1); donde tamaño se refiere a la longitud de la fila o columna de la matriz

	0	1	2	3
0	a	b	c	d
1	1	2	3	4
2	5	6	7	8

- En **Python** la implementación es a través de una lista de listas
- Para acceder a una posición fila y columna se deben utilizar un par de []
- El primer conjunto de corchetes accede a la fila y el segundo a la columna

```
matriz=[['a','b','c','d'],
        [1,2,3,4],
        [5,6,7,8]]
```

```
print(matriz[0][3])#d
print(matriz[1][0])#1
print(matriz[2][3])#8
```

a	0	b	1	c	2	d	3
0		2		3		4	
1		6		7		8	
5							

Diagram illustrating matrix indexing with row and column coordinates:

- (0,0)** points to the first row, first column (cell 'a').
- (0,3)** points to the first row, fourth column (cell 'd').
- (2,0)** points to the third row, first column (cell '5').

- RECORRER UNA MATRIZ

- Generalmente el recorrido de una matriz se hace con el ciclo for anidado, es decir un for dentro de otro for.
- Como son listas de listas disponemos de las mismas funciones para agregar, eliminar y modificar como hemos visto en listas

```
matriz=[["India",1429],
        ["China",1426],
        ["EEUU",340]
        ]

for fila in range(0, len(matriz)):
    for columna in range(0,len(matriz[fila])):
        print("[",fila,"][",columna,"]")
        print("-->",matriz[fila][columna])
```

- CARGA Y VISUALIZACIÓN DE DATOS

CARGA

```
matrizAlumnos=[]

cantidadAlumnos=int(input("Ingrese la cantidad de alumnos que desea cargar"))

for fila in range(0,cantidadAlumnos):
    alumno=[]

    print("Ingrese los datos del alumno ",fila+1)
    nombre=input("Ingrese su nombre")
    legajo= input("Ingrese su legajo")

    alumno.append(nombre)
    alumno.append(legajo)

    matrizAlumnos.append(alumno)
```

VISUALIZACIÓN

```
for fila in range(0, cantidadAlumnos):
    print("Datos del alumno: ",fila+1)
    print("Nombre ",matrizAlumnos[fila][0]," Legajo: ",matrizAlumnos[fila][1])
```

Autoevaluación



Guía de ejercicios Listas



Ejercicio 1

Una tienda registra las ventas de los últimos 7 días.

El programa debe:

1. Guardar las ventas diarias en una lista.
2. Mostrar: el total vendido, el promedio de ventas y el día con mayor venta.



Ejercicio 2

Un sistema meteorológico registra 10 temperaturas diarias.

El programa debe:

1. Guardar las temperaturas en una lista.
2. Informar: cuántas temperaturas superan los 30 grados, la temperatura mínima y la temperatura máxima

Guía de ejercicios Funciones



Ejercicio 3

Un cine tiene 3 filas y 5 asientos por fila.

Se registran con:

- 0 = libre, 1 = ocupado

El programa debe:

1. Cargar la matriz
2. Mostrar: total de asientos ocupados y
total de asientos libres



Ejercicio 4

Una escuela registra las notas de 4 estudiantes en 3 materias.

Las notas se guardan en una matriz.

El programa debe calcular:

- el promedio de cada estudiante

Guía de ejercicios Funciones



Ejercicio 5

Crear una función que calcule el salario semanal de un empleado.

La función debe recibir:

- horas trabajadas
- valor por hora

Debe devolver el salario total.



Ejercicio 6

Crear una función que reciba el precio de un producto.

Si el precio es mayor a 100, aplicar 10% de descuento.

La función debe devolver el precio final.

Guía de ejercicios Funciones



Ejercicio 7

Una escuela registra las notas de 5 estudiantes en 4 materias.

El programa debe:

1. Guardar las notas en una matriz
2. Crear una función que calcule el promedio de un estudiante
3. Mostrar el promedio de cada estudiante.



Ejercicio 8

Un supermercado registra el stock de 4 productos en 3 sucursales.

Los datos se guardan en una matriz.

Crear una función que reciba una lista y calcule el stock total de un producto.

El programa debe mostrar:

- stock total de cada producto.



GRACIAS

Por su atención

